

Riley Rachels

Dr. Xin Wang

CS 3810 Introduction to Artificial Intelligence

2 Oct. 2026

Vacuum World

Design decisions

States were defined as a tuple of position and dirty cells (position, dirty) where position is a tuple coordinate (row, column) and dirty is a frozen set of positions. Dirty had to be a frozen set instead of a set to be hashable when writing the search algorithms. Frozen set achieves this by being immutable, meaning it cannot be altered as a set can; therefore, it is eligible as a dictionary key, which is necessary for hashability. Reopening in A* was handled by the pre-written priority queue. When a duplicate element is pushed into the priority queue, instead of duplicating the entry, the priority decreases.

Heuristic analysis

Both h_1 and h_2 are admissible heuristics, meaning they are never greater than the true cost. h_1 returns the total number of dirty cells left in the environment. The goal of the vacuum is defined by there being no dirty cells remaining; therefore, it is impossible for the vacuum to reach the goal with a cost that is less than the number of dirty cells. Cleaning these cells once reached also adds an additional cost. h_2 returns the sum of the total number of dirty cells and the Manhattan distance to the closest dirty cell. It is impossible for the cost to reach the nearest dirty cell to be less than even the shortest path to the

nearest dirty cell, since there is a cost to clean on arrival and potential obstacles only making the path longer, never shorter.

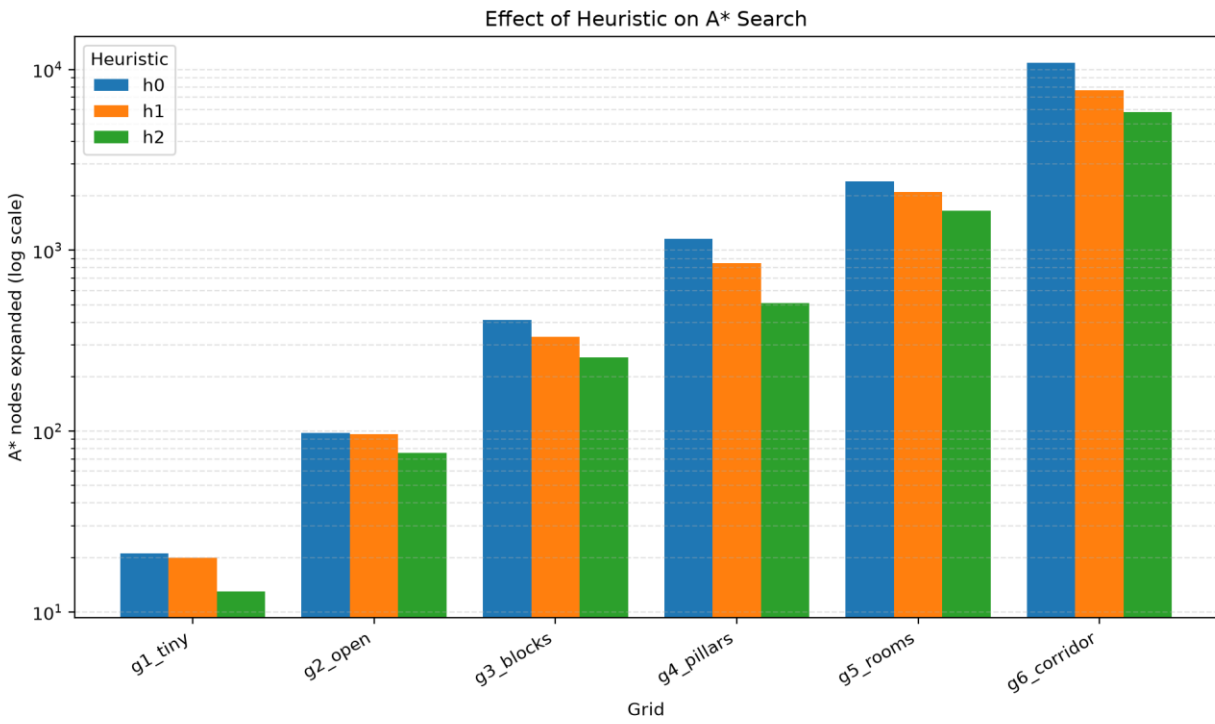
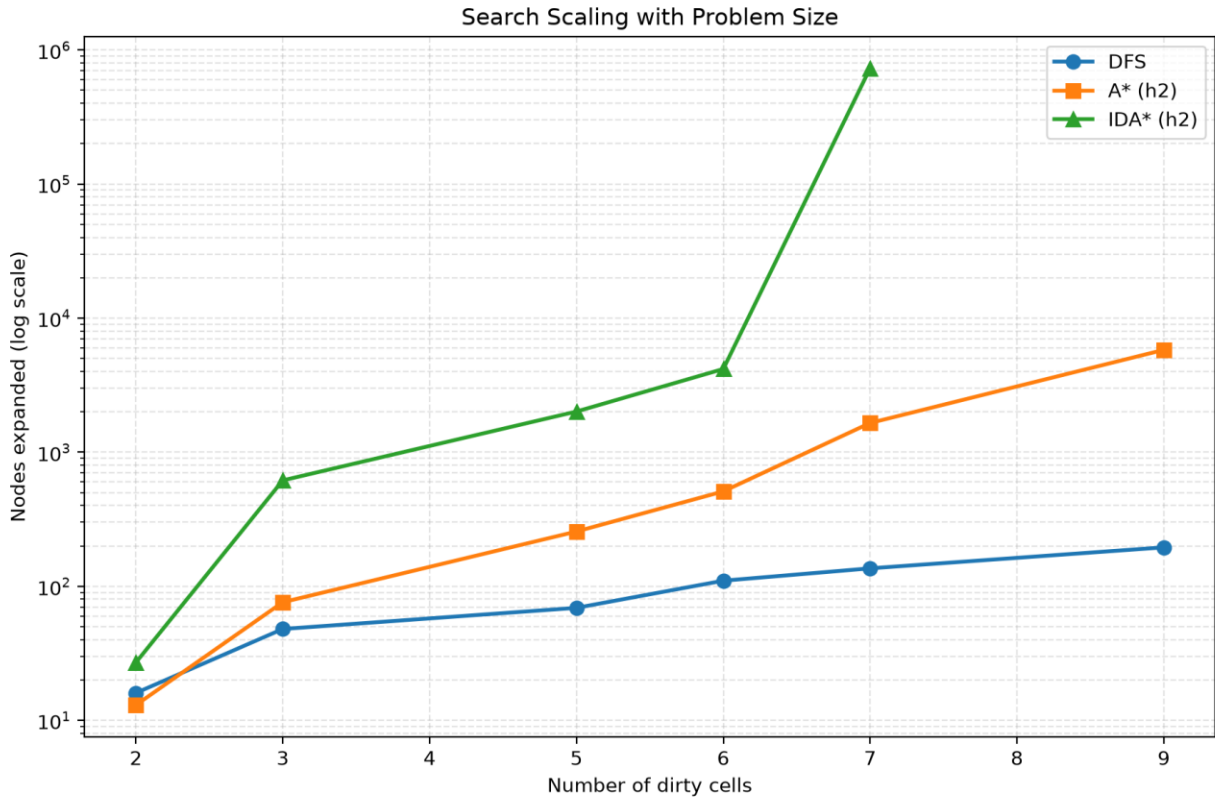
h_2 dominates h_1 , meaning every state's estimated cost in h_2 is greater than or equal to every state's estimated cost in h_1 . The estimated cost in h_2 is the sum of the total number of dirty cells and the Manhattan distance to the nearest dirty cell. Since the estimated cost of h_1 is the total number of dirty cells, this will always be less than or equal to h_2 's estimated cost. A node is expanded when it has been visited, and possible nodes to visit next have been added to the frontier. h_2 dominating h_1 predicts that h_2 will expand fewer or equal nodes to h_1 since h_2 is more specific in its estimation.

h_3 was not attempted.

Experimental results

grid	algorithm	heuristic	status	cost	nodes_expanded	max_frontier	iterations	seconds
g1_tiny	dfs	-	ok	12.0	16.0	5.0	-	0.0001
g1_tiny	astar	h0	ok	7.0	21.0	4.0	-	0.0002
g1_tiny	astar	h1	ok	7.0	20.0	5.0	-	0.0002
g1_tiny	astar	h2	ok	7.0	13.0	4.0	-	0.0002
g1_tiny	astar	h3	SKIP	-	-	-	-	-
g1_tiny	idastar	h0	ok	7.0	95.0	-	8.0	0.0005
g1_tiny	idastar	h1	ok	7.0	52.0	-	6.0	0.0003
g1_tiny	idastar	h2	ok	7.0	27.0	-	4.0	0.0003
g1_tiny	idastar	h3	SKIP	-	-	-	-	-
g2_open	dfs	-	ok	36.0	48.0	32.0	-	0.0003
g2_open	astar	h0	ok	12.0	98.0	17.0	-	0.0007
g2_open	astar	h1	ok	12.0	96.0	18.0	-	0.0007
g2_open	astar	h2	ok	12.0	76.0	21.0	-	0.0007
g2_open	astar	h3	SKIP	-	-	-	-	-
g2_open	idastar	h0	ok	12.0	15586.0	-	13.0	0.1094
g2_open	idastar	h1	ok	12.0	3821.0	-	10.0	0.0233
g2_open	idastar	h2	ok	12.0	616.0	-	4.0	0.0078
g2_open	idastar	h3	SKIP	-	-	-	-	-
g3_blocks	dfs	-	ok	43.0	69.0	34.0	-	0.0006
g3_blocks	astar	h0	ok	15.0	411.0	61.0	-	0.0028
g3_blocks	astar	h1	ok	15.0	331.0	71.0	-	0.0024
g3_blocks	astar	h2	ok	15.0	256.0	87.0	-	0.0029
g3_blocks	astar	h3	SKIP	-	-	-	-	-
g3_blocks	idastar	h0	ok	15.0	29281.0	-	16.0	0.1603
g3_blocks	idastar	h1	ok	15.0	4463.0	-	11.0	0.0256
g3_blocks	idastar	h2	ok	15.0	2010.0	-	8.0	0.0184
g3_blocks	idastar	h3	SKIP	-	-	-	-	-

grid	algorithm	heuristic	status	cost	nodes_expanded	max_frontier	iterations	seconds
g4_pillars	dfs	-	ok	80.0	110.0	50.0	-	0.0008
g4_pillars	astar	h0	ok	19.0	1160.0	161.0	-	0.0078
g4_pillars	astar	h1	ok	19.0	850.0	201.0	-	0.006
g4_pillars	astar	h2	ok	19.0	510.0	191.0	-	0.0049
g4_pillars	astar	h3	SKIP	-	-	-	-	-
g4_pillars	idastar	h0	ok	19.0	58097.0	-	20.0	0.3226
g4_pillars	idastar	h1	ok	19.0	8883.0	-	14.0	0.0518
g4_pillars	idastar	h2	ok	19.0	4174.0	-	12.0	0.0385
g4_pillars	idastar	h3	SKIP	-	-	-	-	-
g5_rooms	dfs	-	ok	92.0	136.0	68.0	-	0.0008
g5_rooms	astar	h0	ok	25.0	2403.0	237.0	-	0.0189
g5_rooms	astar	h1	ok	25.0	2097.0	274.0	-	0.0208
g5_rooms	astar	h2	ok	25.0	1651.0	380.0	-	0.0192
g5_rooms	astar	h3	SKIP	-	-	-	-	-
g5_rooms	idastar	h0	TIMEOUT	-	-	-	-	60.017
g5_rooms	idastar	h1	ok	25.0	1802996.0	-	19.0	8.5972
g5_rooms	idastar	h2	ok	25.0	731040.0	-	17.0	5.7156
g5_rooms	idastar	h3	SKIP	-	-	-	-	-
g6_corridor	dfs	-	ok	121.0	195.0	90.0	-	0.0018
g6_corridor	astar	h0	ok	32.0	10873.0	940.0	-	0.0638
g6_corridor	astar	h1	ok	32.0	7665.0	1066.0	-	0.0445
g6_corridor	astar	h2	ok	32.0	5797.0	1282.0	-	0.0475
g6_corridor	astar	h3	SKIP	-	-	-	-	-
g6_corridor	idastar	h0	TIMEOUT	-	-	-	-	60.0131
g6_corridor	idastar	h1	TIMEOUT	-	-	-	-	60.0205
g6_corridor	idastar	h2	TIMEOUT	-	-	-	-	60.0136
g6_corridor	idastar	h3	SKIP	-	-	-	-	-



Discussion

DFS usually expands fewer nodes due to stopping when it finds any path to the goal. It can only find the optimal solution by getting lucky. Since A* search includes heuristics, it compares paths by their current cost + their estimated cost. Because of this, it can find the optimal solution, but in turn it can expand more nodes due to exploring multiple paths.

For the trials run with A*, the number of nodes expanded goes down as it moves from h0 to h1 to h2. This matches what dominance predicts since h2 uses a more specific heuristic than h1 and h1 than h0. In a more specific heuristic, fewer options are explored because they do not fit the criteria.

IDA* is a good option when memory is limited, such as a vacuum robot. IDA* may expand more nodes than A*, but it uses memory optimally like DFS while inheriting the heuristics of A*. When memory is not an issue for a device, A* is the best choice for its generally greater speed.

As dirty cells are added to the state space, it grows. This is because added dirty cells add cells that must be visited to reach the goal. If a state space has one dirty cell, there is a minimum of one cell that must be visited to meet the goal; when more are added, that minimum increases to the new number of dirty cells. Each dirty cell in a state space has two choices for its state: already cleaned or dirty. Because of this, we can raise 2 to the power of the number of dirty cells. Then this gets multiplied by the number of free cells to get the total number of reachable states.

$$\text{Reachable States} = F * 2^D$$

This “blows up” because adding a dirty cell can double the number of cells that need to be visited, which can get very large.

The G6_Corridor was the hardest grid. It had the largest set of dirty cells, making the number of cells needing to be visited very large, and in one of the rows the grid had obstacles covering all but one entry, making it harder to reach the goal. G6_Corridor was one of two grids that had the result of TIMEOUT, but was the only grid to get this result for all heuristics for the IDA* algorithm.

If the robot in the vacuum world problem did not have unlimited battery life, this would break the heuristics written. The robot may run out of battery before reaching the goal with the current cost estimations. The heuristics would need to be adjusted to account for how many actions total the robot can take before powering off. The robot would need to know when a path is too long for the amount of charge it currently has. It may also need to start from the charging station and account for the number of steps it would take to return to be able to charge before cleaning again.

Reflection

I struggled with getting started setting up the environment for this problem. Having prewritten code, though very helpful, was overwhelming, and it took a lot of rereading and thinking before I could start writing. When writing the IDA* algorithm, it was very challenging to find resources to support implementation. I spent most of my time on IDA* because of this. This was also a challenge for A*, but IDA* more so. It was very easy to find resources on the DFS algorithm. If I did this project again with more time, I would first trace

each algorithm on all or multiple of the test grids to get a better idea of how it works on the specific problem. I tried this with DFS, and it was great for understanding but was very time-consuming and did not make writing the algorithm intuitive. Tutorials and examples were still necessary to complete the implementation, but tracing the algorithm gave me a greater understanding in the context of the specific problem, which helped the implementation click in my mind. I would have gone further with this and also included A* and IDA*.